In [5]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
data=pd.read_csv('C:/Users/Administrator/Downloads/Heart Disease UCI.csv')
data
```

Out[5]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	tl
0	69	1	0	160	234	1	2	131	0	0.1	1	1	
1	69	0	0	140	239	0	0	151	0	1.8	0	2	
2	66	0	0	150	226	0	0	114	0	2.6	2	0	
3	65	1	0	138	282	1	2	174	0	1.4	1	1	
4	64	1	0	110	211	0	2	144	1	1.8	1	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	
292	40	1	3	152	223	0	0	181	0	0.0	0	0	
293	39	1	3	118	219	0	0	140	0	1.2	1	0	
294	35	1	3	120	198	0	0	130	1	1.6	1	0	
295	35	0	3	138	183	0	0	182	0	1.4	0	0	
296	35	1	3	126	282	0	2	156	1	0.0	0	0	

297 rows × 14 columns

In [19]:
```
#first 10 rows
tenrows=data.head(10)
tenrows
```

Out[19]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	69	1	0	160	234	1	2	131	0	0.1	1	1	0
1	69	0	0	140	239	0	0	151	0	1.8	0	2	0
2	66	0	0	150	226	0	0	114	0	2.6	2	0	0
3	65	1	0	138	282	1	2	174	0	1.4	1	1	0
4	64	1	0	110	211	0	2	144	1	1.8	1	0	0
5	64	1	0	170	227	0	2	155	0	0.6	1	0	2
6	63	1	0	145	233	1	2	150	0	2.3	2	0	1
7	61	1	0	134	234	0	0	145	0	2.6	1	2	0
8	60	0	0	150	240	0	0	171	0	0.9	0	0	0
9	59	1	0	178	270	0	2	145	0	4.2	2	0	2

In [10]:
```
#check data types
data.dtypes
```

Out[10]:
```
age            int64
sex            int64
cp             int64
trestbps       int64
chol           int64
fbs            int64
restecg        int64
thalach        int64
exang          int64
oldpeak      float64
slope          int64
ca             int64
thal           int64
condition      int64
dtype: object
```

In [20]:
```
#missing values
missing=data.isna()
missing
```

Out[20]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
0	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...
292	False	False	False	False	False	False	False	False	False	False	False
293	False	False	False	False	False	False	False	False	False	False	False
294	False	False	False	False	False	False	False	False	False	False	False
295	False	False	False	False	False	False	False	False	False	False	False
296	False	False	False	False	False	False	False	False	False	False	False

297 rows × 14 columns

In [21]:
```
#over all summary of the data
summary=data.describe()
summary
```

Out[21]:

	age	sex	cp	trestbps	chol	fbs	reste
count	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	297.000
mean	54.542088	0.676768	2.158249	131.693603	247.350168	0.144781	0.996
std	9.049736	0.468500	0.964859	17.762806	51.997583	0.352474	0.994
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000
25%	48.000000	0.000000	2.000000	120.000000	211.000000	0.000000	0.000
50%	56.000000	1.000000	2.000000	130.000000	243.000000	0.000000	1.000
75%	61.000000	1.000000	3.000000	140.000000	276.000000	0.000000	2.000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000

In [25]:
```
#count number of males
males=data['sex'][data['sex']==1]
males.count()
```

Out[25]: 201

In [26]:
```
#count number of females
females=data['sex'][data['sex']==0]
females.count()
```

Out[26]: 96

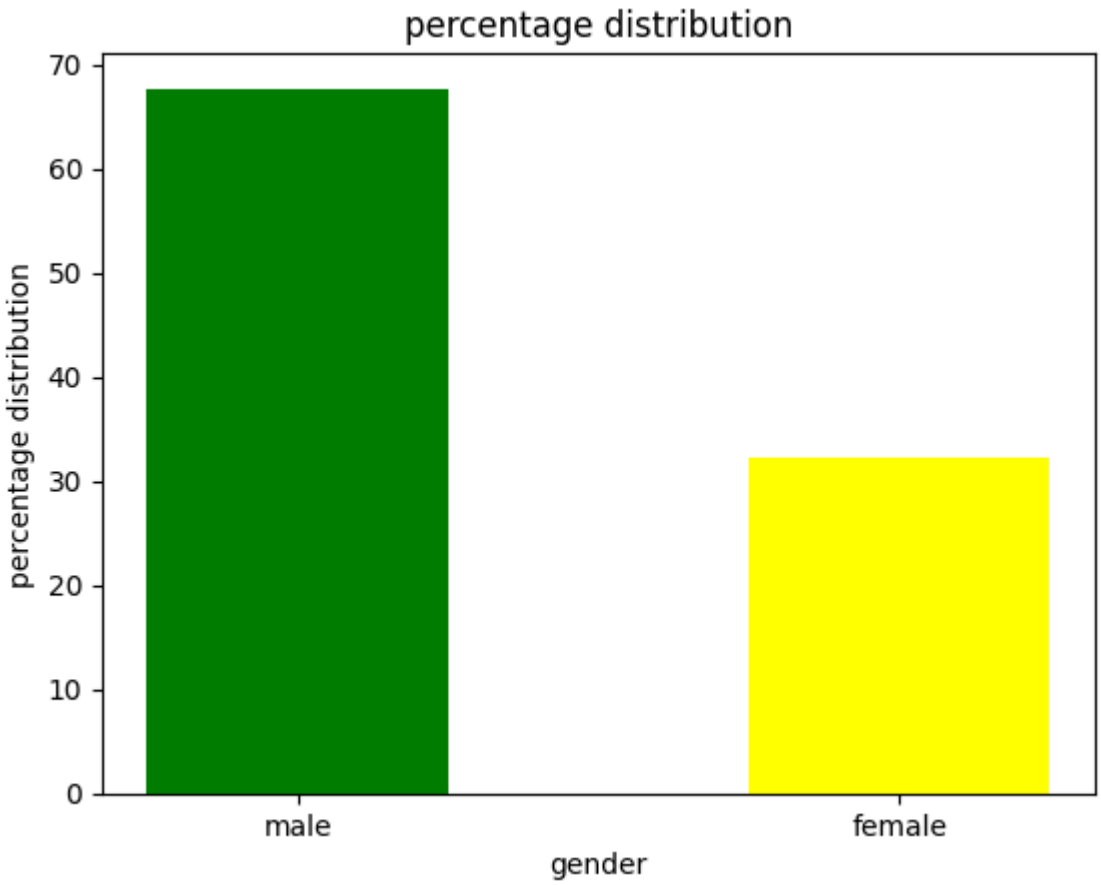
In [34]:
```
#males percentage distribution
male_per=(males.count()/len(data.sex))*100
male_per
```

Out[34]: 67.67676767676768

In [35]:
```
#female percentage distribution
fem_per=(females.count()/len(data.sex))*100
fem_per
```

Out[35]: 32.323232323232325

In [39]:
```
#bar chart for per distribution
gen=['male','female']
coun=[male_per,fem_per]
plt.bar(gen,coun,color=['green','yellow'],width=0.5)
plt.xlabel('gender')
plt.ylabel('percentage distribution')
plt.title('percentage distribution')
plt.show()
```



In [43]:
```
#find minimum age
min1=data['age'].min()
min1
```

Out[43]: 29

In [44]:
```
#maximum age f
max1=data['age'].max()
max1
```

Out[44]: 77

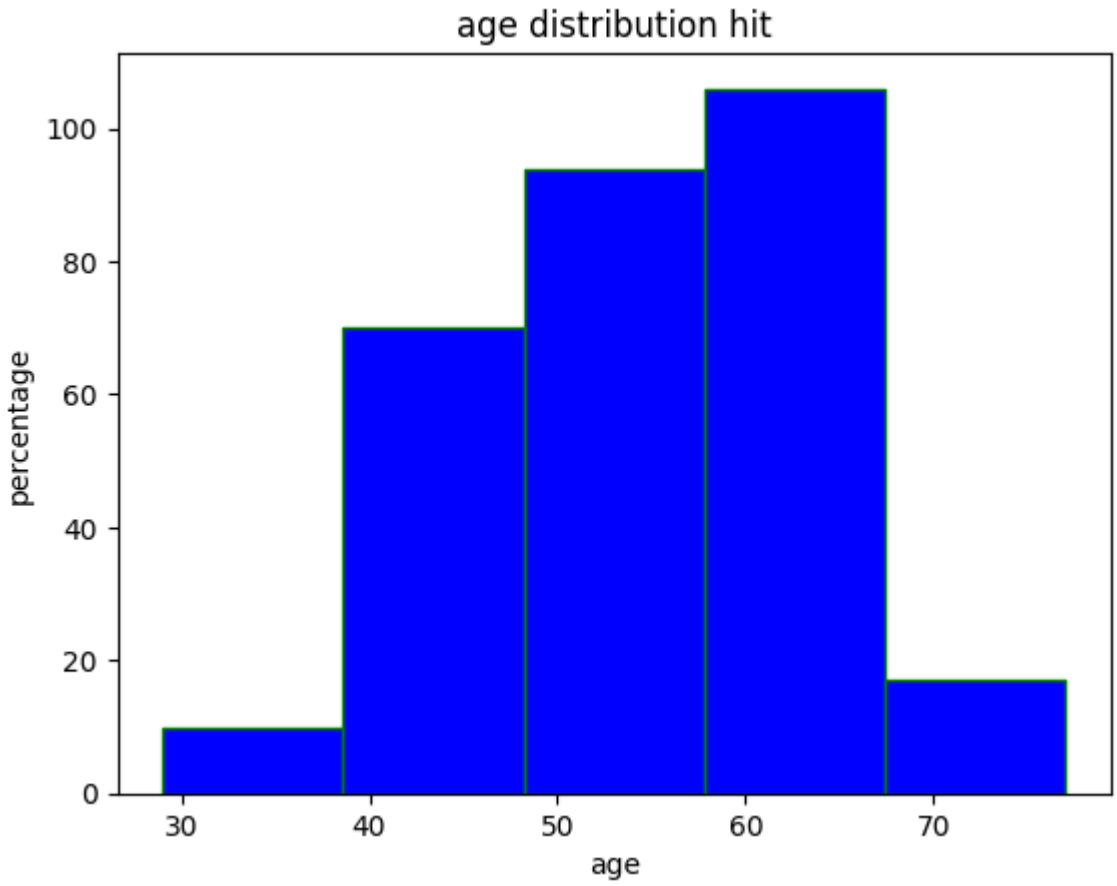
In [45]:
```
#mean age dis
mean1=data['age'].mean()
mean1
```

Out[45]: 54.54208754208754

In [48]:
```
#median age dis
meadian1=data['age'].median()
meadian1
```

Out[48]: 56.0

In [54]:
```
#hist of age distribution
plt.hist(data.age,color='blue',edgecolor='green',bins=5)
plt.xlabel('age')
plt.ylabel('percentage')
plt.title('age distribution hit')
plt.show()
```



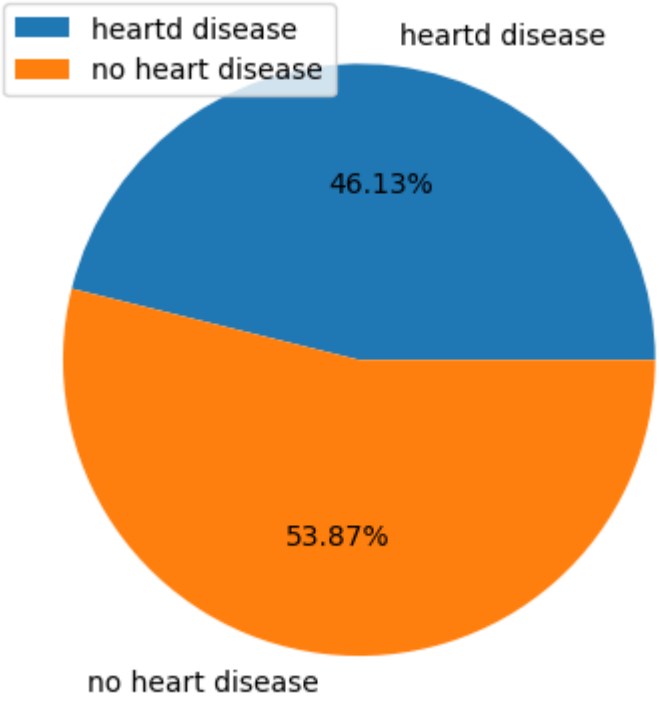
In [56]:
```
#count number of hear disease
hd=data['condition'][data['condition']==1]
hdc=hd.count()
hdc
```

Out[56]: 137

In [58]:
```
#count number of no heart disease
hdn=data['condition'][data['condition']==0]
hdnc1=hdn.count()
hdnc1
```

Out[58]: 160

In [60]:
```
#plot pie chart
phd=[hdc,hdnc1]
name=['heartd disease','no heart disease']
plt.pie(phd,labels=name,autopct='%2.2f%%')
plt.legend()
plt.show()
```



In [62]:
```
#calculate disease percentage
cdp=(hd.count()/len(data.condition)*100)
cdp
```

Out[62]: 46.12794612794613

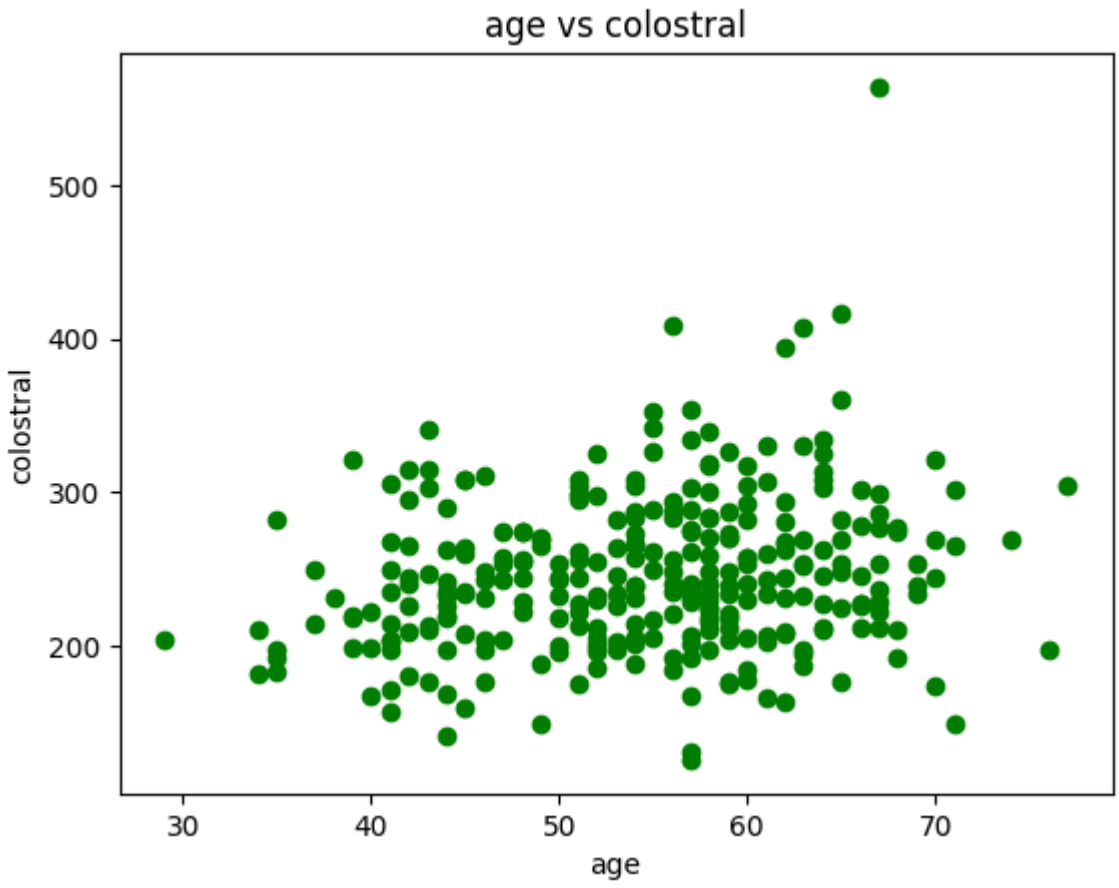
In [63]:
```
#Calculate correlation using df.corr()
corre=data.corr()
corre
```

Out[63]:

	age	sex	cp	trestbps	chol	fbs	restecg	
age	1.000000	-0.092399	0.110471	0.290476	0.202644	0.132062	0.149917	-(
sex	-0.092399	1.000000	0.008908	-0.066340	-0.198089	0.038850	0.033897	-(
cp	0.110471	0.008908	1.000000	-0.036980	0.072088	-0.057663	0.063905	-(
trestbps	0.290476	-0.066340	-0.036980	1.000000	0.131536	0.180860	0.149242	-(
chol	0.202644	-0.198089	0.072088	0.131536	1.000000	0.012708	0.165046	-(
fbs	0.132062	0.038850	-0.057663	0.180860	0.012708	1.000000	0.068831	-(
restecg	0.149917	0.033897	0.063905	0.149242	0.165046	0.068831	1.000000	-(
thalach	-0.394563	-0.060496	-0.339308	-0.049108	-0.000075	-0.007842	-0.072290	1
exang	0.096489	0.143581	0.377525	0.066691	0.059339	-0.000893	0.081874	-(
oldpeak	0.197123	0.106567	0.203244	0.191243	0.038596	0.008311	0.113726	-(
slope	0.159405	0.033345	0.151079	0.121172	-0.009215	0.047819	0.135141	-(
ca	0.362210	0.091925	0.235644	0.097954	0.115945	0.152086	0.129021	-(
thal	0.120795	0.370556	0.266275	0.130612	0.023441	0.051038	0.013612	-(
condition	0.227075	0.278467	0.408945	0.153490	0.080285	0.003167	0.166343	-(

In [64]:

```
#scatter plot for agevs colestrol
col=data.chol
age1=data.age
plt.scatter(age1,col,marker='o',color='green')
plt.xlabel('age')
plt.ylabel('colostral')
plt.title('age vs colostral')
plt.show()
```



In [6]:
```
#group by cp
gcp=data.groupby('cp')
gcp
```

Out[6]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001E14D862FC0>
```

In [11]:
```
#disease rate all
dsr=gcp['condition'].mean().reset_index()*100
dsr
```

Out[11]:

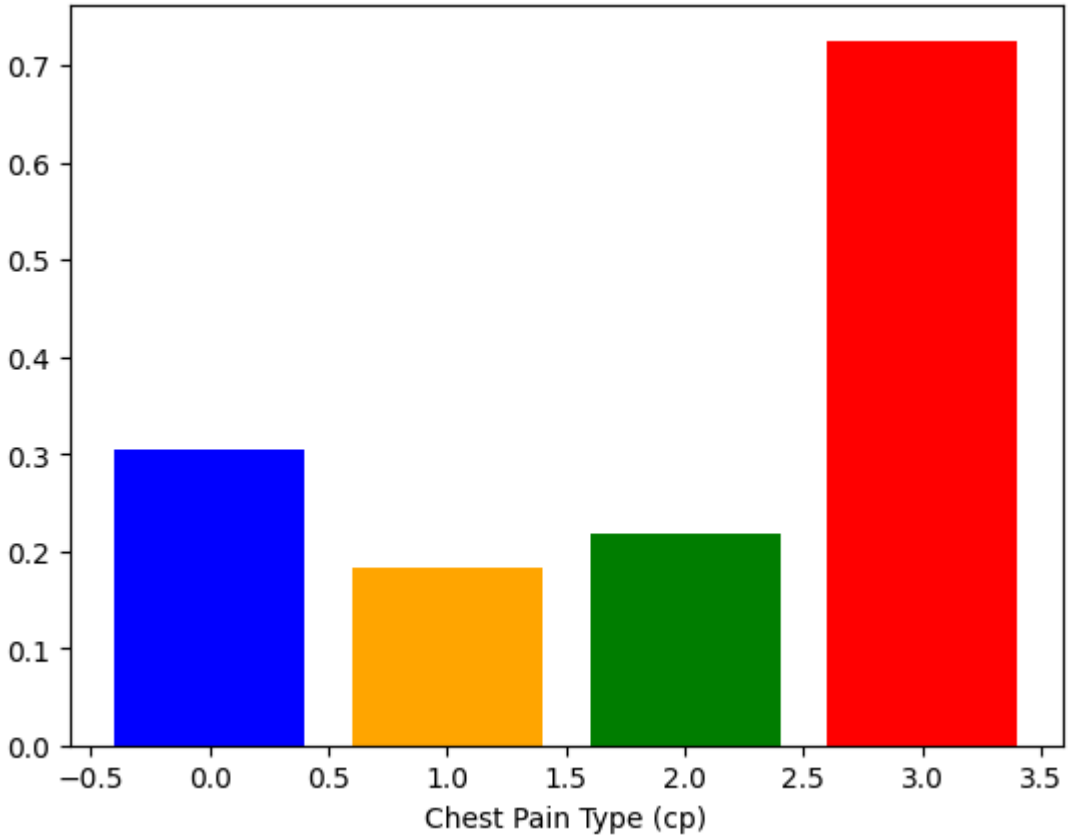
	cp	condition
0	0	30.434783
1	100	18.367347
2	200	21.686747
3	300	72.535211

In [12]:
```
maxc=dsr.max()
maxc
```

Out[12]:
```
cp            300.000000
condition      72.535211
dtype: float64
```

In [73]:
```
#bar chart grouped
gro = data.groupby('cp')['condition'].mean().reset_index()
plt.bar(gro['cp'],gro['condition'], color=['blue','orange','green','red'])
```

```
plt.xlabel('Chest Pain Type (cp)')
plt.show()
```



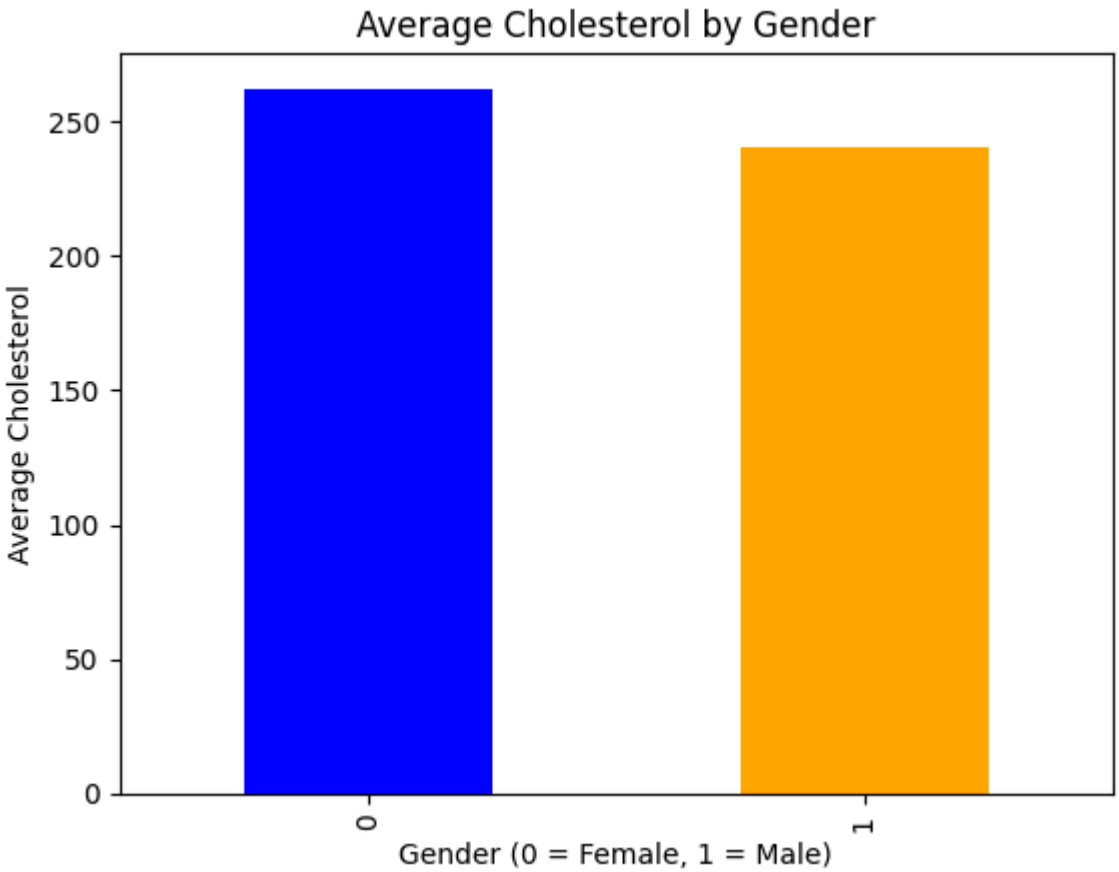
In [86]:
```
#group the data by gender
grocol= data.groupby('sex')['chol']
grocol
```

Out[86]:
```
<pandas.core.groupby.generic.SeriesGroupBy object at 0x000002744708F920>
```

In [85]:
```
#average colostral by geander wise
avgcolgen= data.groupby('sex')['chol'].mean()
avgcolgen
```

Out[85]:
```
sex
0    262.229167
1    240.243781
Name: chol, dtype: float64
```

In [90]:
```
#Visualize using bar plot
avgcolgen.plot(kind='bar', color=['blue','orange'])
plt.xlabel('Gender (0 = Female, 1 = Male)')
plt.ylabel('Average Cholesterol')
plt.title('Average Cholesterol by Gender')
plt.show()
```



In [91]:
```
#Resting Blood Pressure Analysis
#Find:
#Average BP
avgbp=data['trestbps'].mean()
avgbp
```

Out[91]: 131.69360269360268

In [92]:
```
#Patients with BP > 140
bpg=data[data['trestbps']>140]
bpg
```

Out[92]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	tl
0	69	1	0	160	234	1	2	131	0	0.1	1	1	
2	66	0	0	150	226	0	0	114	0	2.6	2	0	
5	64	1	0	170	227	0	2	155	0	0.6	1	0	
6	63	1	0	145	233	1	2	150	0	2.3	2	0	
8	60	0	0	150	240	0	0	171	0	0.9	0	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	
263	50	1	3	150	243	0	2	128	0	2.6	1	0	
264	50	1	3	144	200	0	2	126	1	0.9	1	0	
277	45	1	3	142	309	0	2	147	1	0.0	1	3	
285	43	1	3	150	247	0	0	171	0	1.5	0	0	
292	40	1	3	152	223	0	0	181	0	0.0	0	0	

66 rows × 14 columns

In [95]:

```
#disease prescence in high bp group
disease_rate_high_bp =bpg['condition'].mean()
disease_rate_all=data['condition'].mean()
disease_rate_high_bp
disease_rate_all
```

Out[95]:

```
0.4612794612794613
```

In [96]:

```
#Maximum Heart Rate vs Disease
#Compare average thalach for:
#Disease patients
disease_patients=data[data['condition']==1]['thalach']
disease_patients
```

Out[96]:

```
3      174
7      145
10     159
11     125
12     162
      ...
291    114
292    181
293    140
294    130
296    156
Name: thalach, Length: 137, dtype: int64
```

In [97]:

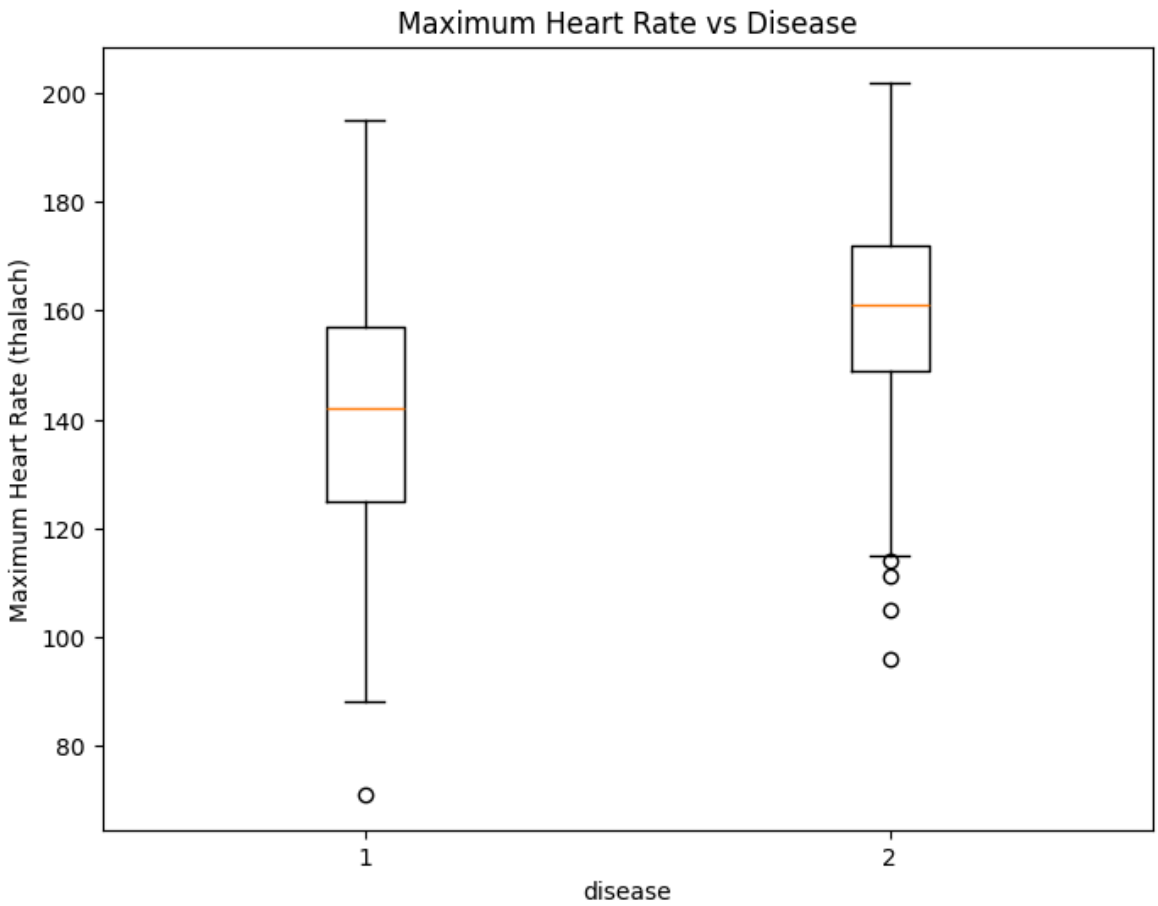
```
#non disease patients
non_disease_patients=data[data['condition']==0]['thalach']
non_disease_patients
```

Out[97]:
```
0      131
1      151
2      114
4      144
5      155
      ...
285    171
286    161
287    122
289    178
295    182
Name: thalach, Length: 160, dtype: int64
```

In [98]:
```
#average
avg_disease=disease_patients.mean()
avg_non_disease=non_disease_patients.mean()
avg_disease
avg_non_disease
```

Out[98]:
```
158.58125
```

In [101]:
```
#box plot show
patients=[disease_patients, non_disease_patients]
plt.figure(figsize=(8,6))
plt.boxplot(patients)
plt.xlabel('disease')
plt.ylabel('Maximum Heart Rate (thalach)')
plt.title('Maximum Heart Rate vs Disease')
plt.show()
```



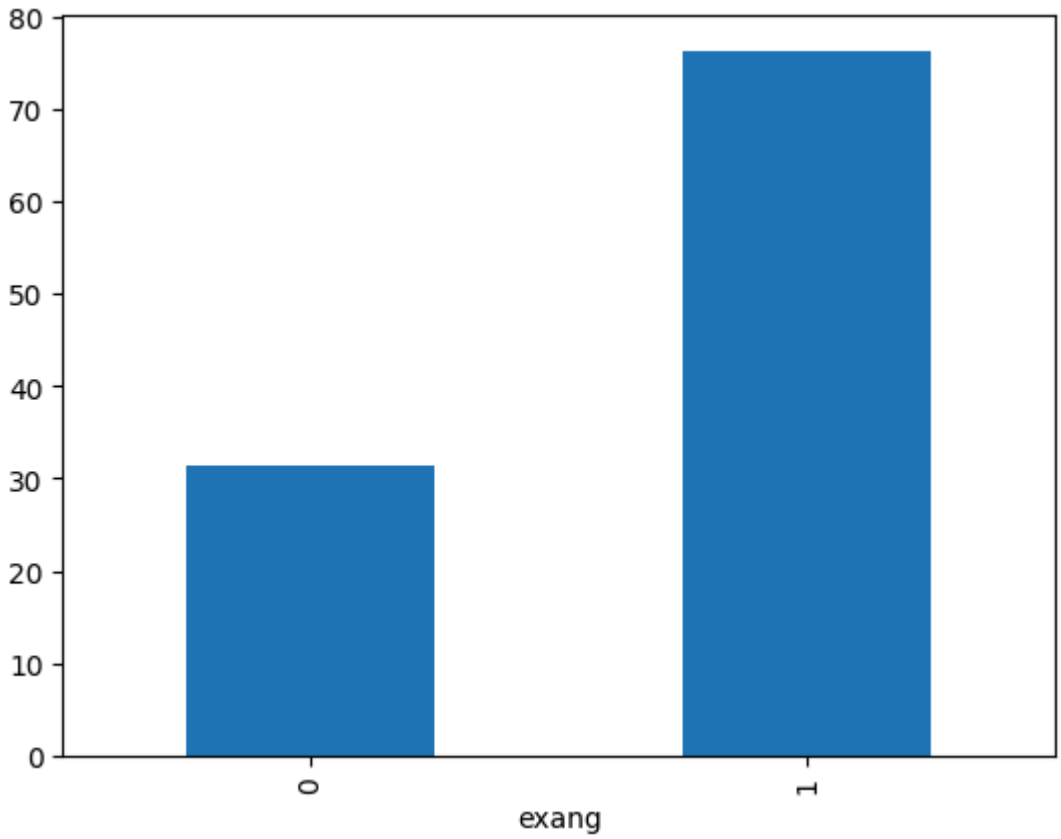
In [105]:
```
#Calculate disease percentage
#exang induced agena
exang1=data['exang'][data['exang']==1]
perexang1=(exang1.count()/len(data.exang))*100
perexang1
```

Out[105]:
```
32.659932659932664
```

In [106]:
```
#exang induceds agena
exang0=data['exang'][data['exang']==0]
perexang0=(exang0.count()/len(data.exang))*100
perexang0
```

Out[106]:
```
67.34006734006735
```

In [115]:
```
#visulize bar chart
exp=data.groupby('exang')['condition'].mean()*100
exp.plot(kind='bar')
plt.show()
```

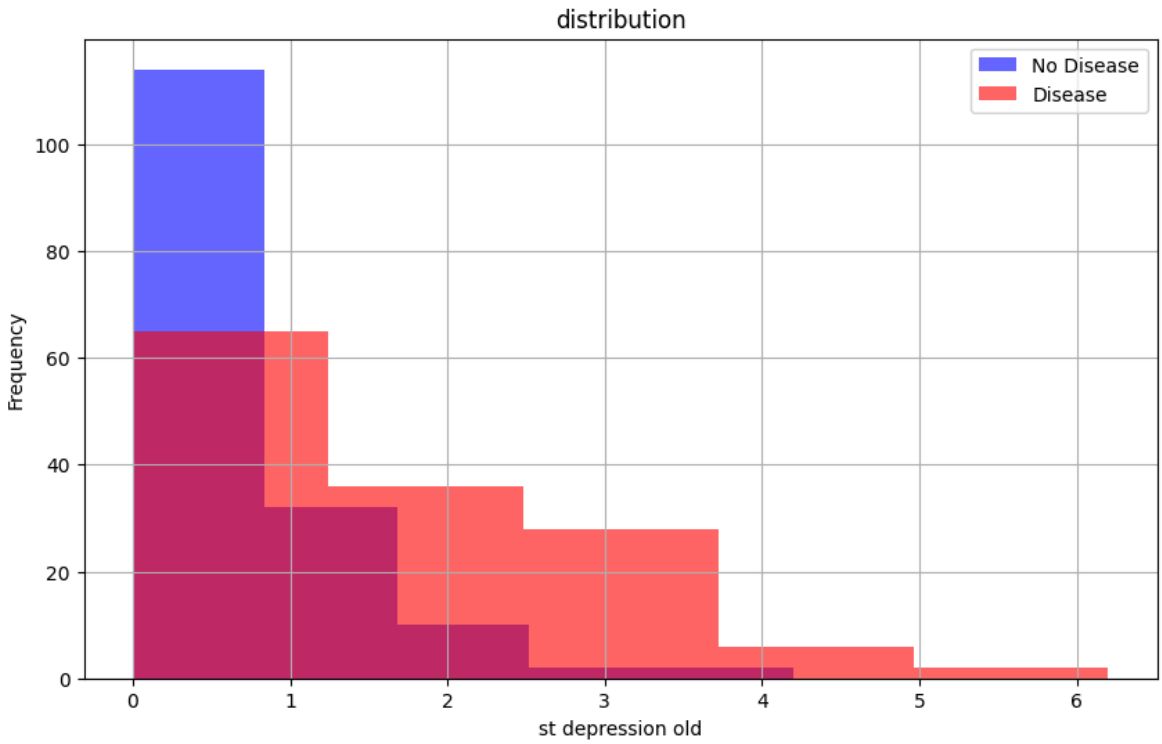


In [116]:
```
#st depression old peak analysis
#calculate old mean peak by condition
oldpeakmen=data.groupby('condition')['oldpeak'].mean()
oldpeakmen
```

Out[116]:
```
condition
0    0.598750
1    1.589051
Name: oldpeak, dtype: float64
```

In [117]:
```
#plot
plt.figure(figsize=(10,6))
his0=data[data['condition']==0]['oldpeak']
```

```
his0.hist(alpha=0.6, color='blue', bins=5,label='No Disease')
his1=data[data['condition']==1]['oldpeak']
his1.hist(alpha=0.6, color='red', bins=5,label='Disease')
plt.xlabel('st depression old')
plt.ylabel('Frequency')
plt.title('distribution')
plt.legend()
plt.show()
```



In [118]:
```
#Number of Major Vessels ca
#Group by ca
gca=data.groupby('ca')
gca
```

Out[118]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x0000027449781670>
```

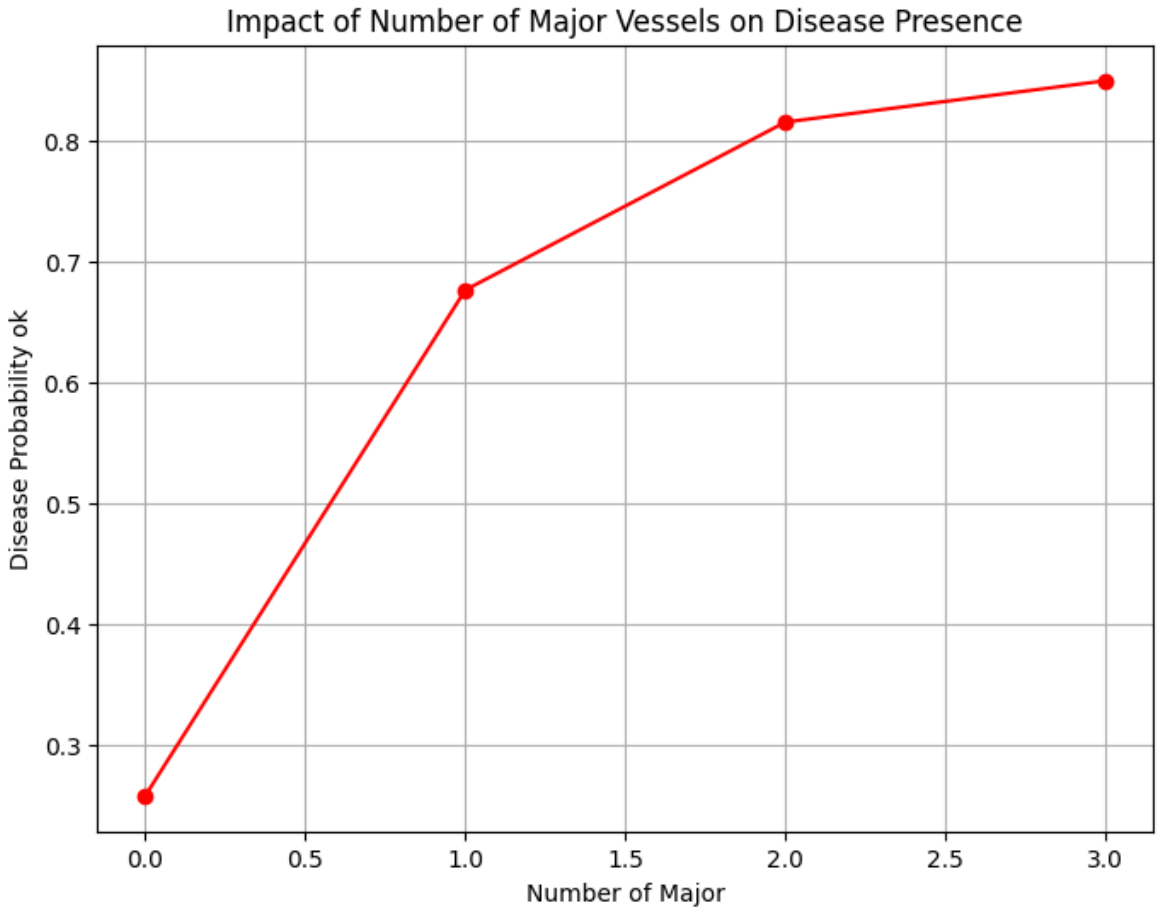
In [119]:
```
#calculate disease probabilty
gca1= data.groupby('ca')['condition'].mean().reset_index()
gca1
```

Out[119]:

	ca	condition
0	0	0.258621
1	1	0.676923
2	2	0.815789
3	3	0.850000

In [120]:
```
#plot the graph
plt.figure(figsize=(8,6))
plt.plot(gca1['ca'],gca1['condition'],marker='o',linestyle='-',color='red')
plt.xlabel('Number of Major')
plt.ylabel('Disease Probability ok')
plt.title('Impact of Number of Major Vessels on Disease Presence')
```

```
plt.grid(True)
plt.show()
```



In [122]:
```
#Thalassemia vs Disease
#Cross-tabulate thal and target
crosstab = pd.crosstab(data['thal'],data['condition'])
crosstab
```

Out[122]:

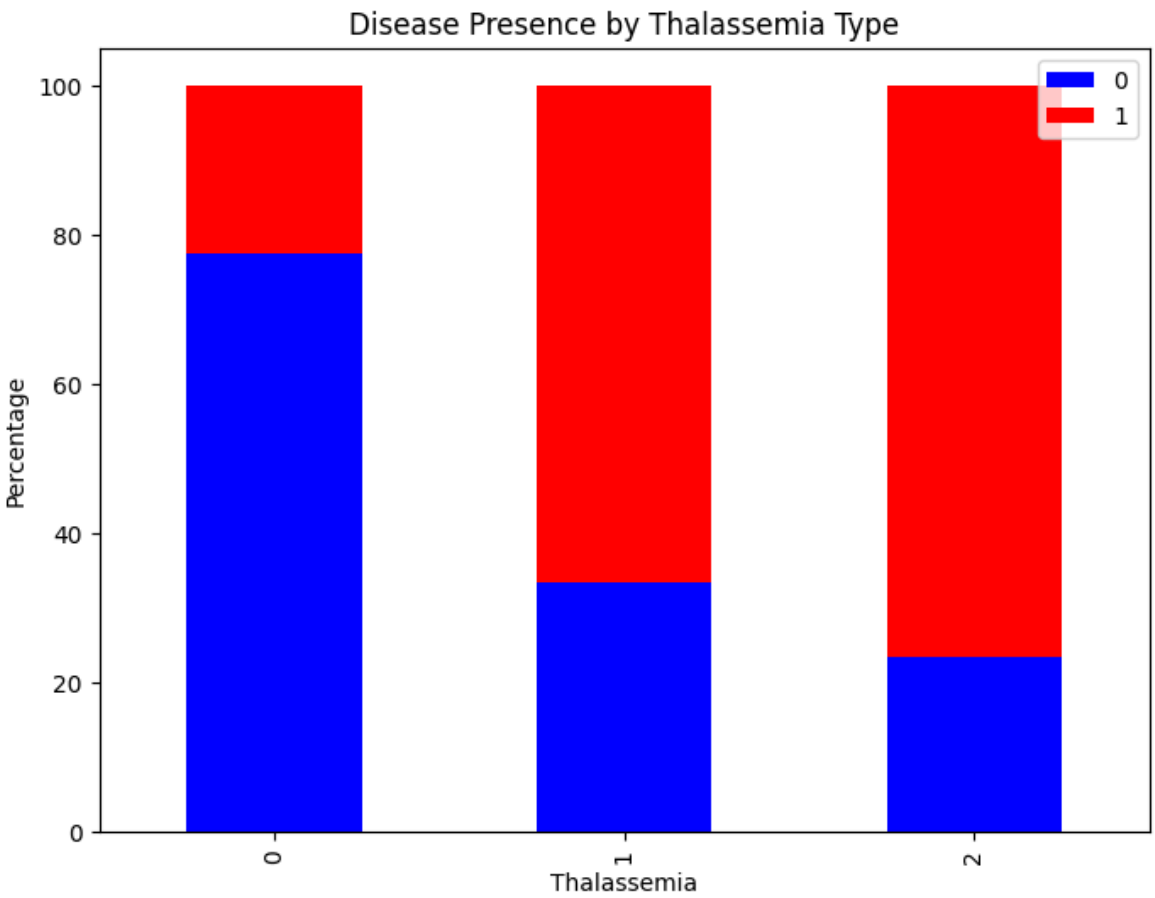
condition	0	1
thal		
0	127	37
1	6	12
2	27	88

In [123]:
```
#conver to percentage
crosstab_pct=crosstab.div(crosstab.sum(axis=1), axis=0) * 100
crosstab
```

Out[123]:

condition	0	1
thal		
0	127	37
1	6	12
2	27	88

In [126]:
```
#plot stach barchat
crosstab_pct.plot(kind='bar', stacked=True, figsize=(8,6), color=['blue','red'])
plt.xlabel('Thalassemia')
plt.ylabel('Percentage')
plt.title('Disease Presence by Thalassemia Type')
plt.legend()
plt.show()
```

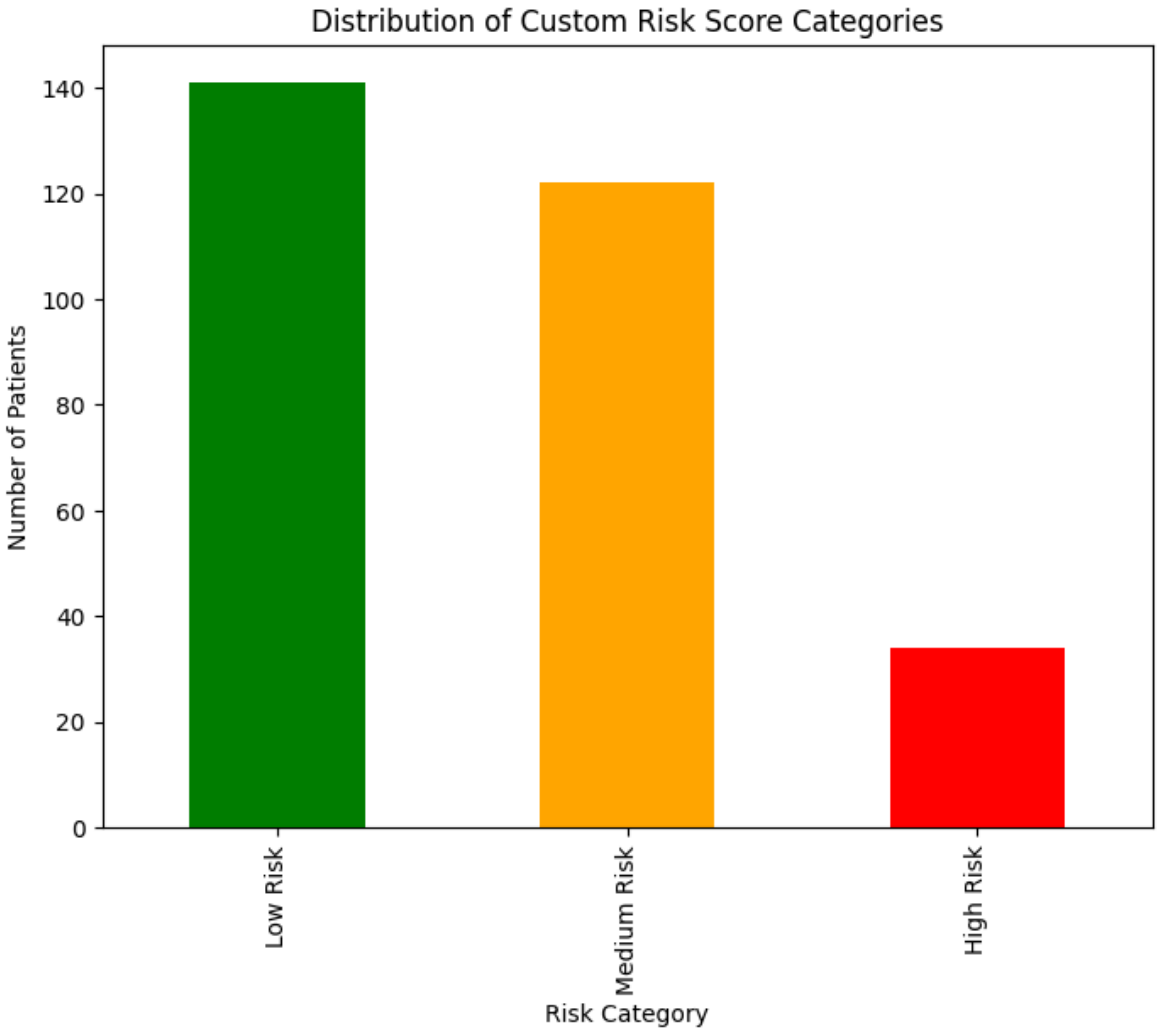


In [131]:
```
#multi factor risk analysis on
age=data['age'].values
chol=data['chol'].values
bp=data['trestbps'].values
condition=data['condition'].values
age
chol
bp
condition
m1=(age > 50) & (chol > 240) & (bp > 140)
subset_targets=condition[m1]
disease_percentage=(np.sum(subset_targets== 1)/len(subset_targets))*100
disease_percentage
```

Out[131]: 66.66666666666666

In [135]:
```
#create risk score
data['risk_score'] = (data['chol'] / 200) + (data['trestbps'] / 120) + data['oldpe
def classify_risk(score):
    if score < 3:
        return 'Low Risk'
    elif score < 5:
        return 'Medium Risk'
```

```
    else:
        return 'High Risk'
data['risk_category'] = data['risk_score'].apply(classify_risk)
data['risk_category'].value_counts()
plt.figure(figsize=(8,6))
data['risk_category'].value_counts().plot(kind='bar', color=['green','orange','red
plt.xlabel('Risk Category')
plt.ylabel('Number of Patients')
plt.title('Distribution of Custom Risk Score Categories')
plt.show()
```



In [ ]:
```
1)Does cholesterol strongly impact heart disease?
Yes, cholesterol strongly impacts heart disease.
2)Is male population more vulnerable?
Yes, males are generally more vulnerable to heart disease.
3)Does exercise-induced angina significantly increase risk?
Yes, exercise-induced angina significantly increases risk.
4)Which feature has strongest correlation with disease?
Cholesterol shows the strongest correlation with disease.
```